



Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Sistema Multiagente para Cuidados de Saúde

Trabalho Prático

Grupo 7

Bruno Alves	Fernando Pires	Lara Pereira	Pedro Teixeira	Sara Silva
pg55924	pg60253	pg57884	pg60294	pg61546

Data da Receção	
Responsável	
Avaliação	
Observações	

Sistema Multiagente para Cuidados de Saúde

Bruno Alves
pg55924

Fernando Pires
pg60253

Lara Pereira
pg57884

Pedro Teixeira
pg60294

Sara Silva
pg61546

janeiro, 2026

Índice

1. Introdução	1
1.1. Objetivos	1
2. Arquitetura do Sistema	2
3. Comunicação entre Agentes	3
3.1. Mensagens	3
3.2. Comunicação	4
3.3. <i>Templates</i>	5
4. Domínios do Sistema	6
4.1. Gestão de Emergências	6
4.1.1. EmergencyCenterAgent	6
4.1.2. TriageAgent	7
4.1.3. AmbulanceAgent	7
4.1.4. HelicopterAgent	7
4.1.5. HospitalAgent	8
4.2. Suporte ao Paciente	8
4.2.1. ChatbotPatientAgent	8
4.3. Agendamentos	9
4.3.1. SchedulerAgent	10
4.3.2. DoctorAgent	10
4.4. Monitorização	11
4.4.1. MonitoringAgent	11
4.4.2. SensorAgent	12
5. Políticas de Decisão	13
5.1. Triagem no Suporte ao Paciente (<i>ChatbotPatientAgent</i>)	13
5.2. Critérios Clínicos de Triagem (<i>TriageAgent</i>)	13
5.3. Detecção Automática de Situações Críticas (<i>MonitoringAgent</i>)	14
5.4. Gestão de Recursos e Resiliência a Indisponibilidades	15
5.5. Continuidade de Cuidados e Encaminhamento para Especialista	16
6. Fluxos de Funcionamento	18
6.1. Fluxo 1: Pedido de Consulta Médica	18
6.2. Fluxo 2: Emergência Detetada por Monitorização	18
6.3. Fluxo 3: Emergência Reportada por Paciente	19
6.4. Fluxo 4: Encaminhamento Hospitalar e Continuidade de Cuidados	20
7. Análise Crítica	21
7.1. Resultados Obtidos	21
8. Conclusões	23
9. Avaliação Por Pares	24

Lista de Figuras

Figura 1 Arquitetura do Sistema Multiagente desenvolvido

2

1. Introdução

O presente projeto tem como objetivo a concepção e implementação de um sistema multiagente aplicado ao domínio dos cuidados de saúde. Este trabalho insere-se no âmbito da Unidade Curricular de Agentes e Sistemas Multiagente e pretende consolidar os conhecimentos relacionados com a modelação, coordenação e comunicação entre agentes autónomos dentro de um ambiente distribuído.

O domínio dos cuidados de saúde caracteriza-se por uma elevada complexidade, dinamismo e necessidade de resposta em tempo útil, especialmente em contextos como a gestão de emergências médicas, o acompanhamento de pacientes e a monitorização contínua de dados clínicos. Neste contexto, os sistemas multiagente surgem como uma abordagem particularmente adequada, uma vez que permitem a distribuição de responsabilidades por entidades autónomas, capazes de perceber o ambiente, tomar decisões locais e cooperar entre si para atingir objetivos globais. A utilização de agentes inteligentes neste domínio contribui para sistemas mais escaláveis, robustos e adaptáveis a diferentes cenários clínicos e operacionais.

O sistema desenvolvido concretiza estes princípios através de uma arquitetura distribuída e modular, organizada em domínios funcionais interligados. Cada domínio é composto por agentes especializados, implementados com a *framework* SPADE, que comunicam entre si através de mensagens assíncronas com performatives inspiradas na norma FIPA-ACL. Esta abordagem permite simular cenários realistas de coordenação clínica, mantendo o sistema modular e extensível.

1.1. Objetivos

Este projeto propõe-se a desenvolver um sistema multiagente funcional e extensível, orientado à simulação e coordenação de diferentes processos no contexto clínico e hospitalar. O sistema integra múltiplos agentes autónomos que cooperam entre si para suportar atividades de resposta a eventos, gestão de recursos e prestação de serviços médicos, de forma distribuída.

Neste sentido, o sistema inclui os seguintes objetivos:

Objetivo 1: Simular a alocação inteligente de recursos de emergência, considerando a disponibilidade e o tipo de meio (ambulância ou helicóptero) mais adequado para cada situação reportada;

Objetivo 2: Gerir a capacidade e o estado das unidades hospitalares, permitindo que agentes hospitalares comuniquem a sua disponibilidade para receber pacientes em contexto de emergência;

Objetivo 3: Desenvolver um sistema de agendamento de consultas médicas suportado por agentes especializados, capazes de coordenar horários entre médicos e pacientes de forma autónoma;

Objetivo 4: Implementar agentes médicos com comportamentos próprios, responsáveis por aceitar, rejeitar ou reorganizar marcações de consultas com base na sua disponibilidade;

Objetivo 5: Criar um subsistema de suporte ao paciente através de um *chatbot*, capaz de interagir com utilizadores, recolher informação inicial e encaminhar pedidos de forma adequada;

Objetivo 6: Integrar mecanismos de triagem automática, permitindo ao sistema distinguir entre situações de emergência, pedidos de apoio geral ou simples esclarecimentos, encaminhando-os para os agentes competentes;

Objetivo 7: Simular a monitorização contínua de parâmetros de saúde, através de agentes responsáveis pela recolha, análise e comunicação de dados relevantes sobre o estado dos pacientes.

2. Arquitetura do Sistema

A arquitetura do sistema desenvolvido segue uma abordagem modular e distribuída, baseada em múltiplos agentes autônomos que cooperam entre si para suportar diferentes funcionalidades no domínio dos cuidados de saúde. O sistema encontra-se organizado em domínios funcionais bem definidos, cada um responsável por um subconjunto específico de tarefas.

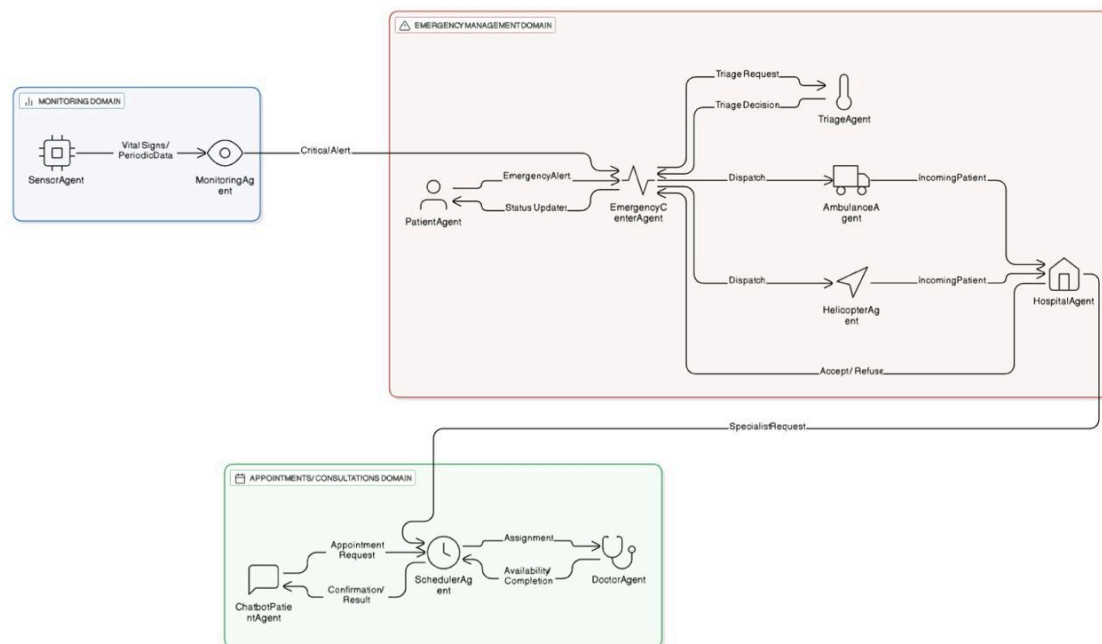


Figura 1: Arquitetura do Sistema Multiagente desenvolvido

De forma global, a arquitetura assenta sobre uma infraestrutura de comunicação comum, suportada por um servidor XMPP (OpenFire), através do qual todos os agentes comunicam utilizando mensagens assíncronas, de acordo com os princípios da framework SPADE. A comunicação entre agentes recorre a performatives inspiradas na norma FIPA-ACL, garantindo clareza semântica e interoperabilidade entre os diferentes componentes do sistema.

O sistema está estruturado em três domínios funcionais interligados, cada um composto por agentes com responsabilidades bem definidas. O domínio de monitorização é responsável pela recolha periódica de sinais vitais através de agentes sensores e pela análise desses dados, podendo desencadear alertas críticos. Sempre que é detetada uma situação de risco, este domínio comunica diretamente com o domínio de gestão de emergências.

O domínio de gestão de emergências assume um papel central na arquitetura, sendo responsável pela receção de alertas, triagem dos casos, despacho de meios de transporte e articulação com unidades hospitalares. Este domínio coordena a resposta global a situações críticas, interagindo tanto com agentes de transporte como com agentes hospitalares.

O domínio de agendamentos e consultas é responsável pela gestão de pedidos de marcação, pela coordenação entre pacientes e médicos e pela atribuição de especialidades quando necessário. Este domínio integra ainda um agente de suporte ao paciente, sob a forma de um *chatbot*, que atua como ponto de entrada para pedidos de consulta e esclarecimentos gerais.

O agente hospitalar desempenha um papel transversal na arquitetura, interligando os domínios de emergências e de agendamentos. Para além de gerir admissões em contexto de emergência, pode solicitar encaminhamentos para consultas especializadas, promovendo a continuidade do cuidado ao paciente entre diferentes fluxos do sistema.

3. Comunicação entre Agentes

A comunicação entre os agentes do sistema baseia-se num modelo de colaboração distribuída, suportado pela *framework* SPADE e por um servidor XMPP. Neste modelo, os agentes interagem exclusivamente através de mensagens assíncronas, mantendo baixo acoplamento entre componentes e permitindo que o sistema seja escalável, modular e resiliente a falhas locais.

A semântica da comunicação segue os princípios definidos pela norma FIPA-ACL (*Agent Communication Language*), na qual cada mensagem expressa explicitamente a intenção do agente emissor, permitindo que os agentes interpretem corretamente pedidos, respostas e atualizações de estado.

3.1. Mensagens

Cada mensagem trocada no sistema é composta por três elementos fundamentais:

- **Performative (intenção):** define o objetivo da mensagem no contexto da interação entre agentes. No sistema desenvolvido são utilizadas as performatives `REQUEST`, `INFORM`, `CONFIRM` e `REFUSE`, correspondendo respetivamente a pedidos de ação, envio de informação, confirmação de execução e rejeição explícita de um pedido.

A definição destas *performatives* encontra-se centralizada no módulo de protocolos do sistema, conforme ilustrado no excerto seguinte:

```
class Performative(str, Enum):  
    REQUEST = "request"  
    INFORM = "inform"  
    CONFIRM = "confirm"  
    REFUSE = "refuse"
```

- **Tipo de mensagem (*msg_type*):** identifica o evento ou ação específica no domínio em questão, como por exemplo alertas de emergência, pedidos de consulta ou atualizações de sinais vitais.
- **Conteúdo (*payload*):** informação útil associada à mensagem, geralmente estruturada em formato `JSON`, que contém dados relevantes como identificadores, especialidades médicas, sinais vitais ou níveis de severidade.

A construção e interpretação das mensagens é realizada através de funções utilitárias comuns, garantindo uma estrutura uniforme em todo o sistema:

```
def build_spade_message(to, performative, msg_type, payload):  
    msg = Message(to=to)  
    msg.set_metadata("performative", performative)  
    msg.set_metadata("type", msg_type)  
    msg.body = encode_payload(payload)  
    return msg  
  
def parse_spade_message(msg):  
    performative = msg.get_metadata("performative")  
    msg_type = msg.get_metadata("type")  
    payload = decode_payload(msg.body)  
    return performative, msg_type, payload, msg.metadata
```

De forma a garantir consistência semântica e evitar a utilização dispersa de *strings* ao longo do código, os tipos de mensagem foram centralizados em *enums* específicos por domínio funcional, tais como

emergências, agendamentos, monitorização e suporte clínico. Esta abordagem reduz erros de integração e facilita a manutenção e extensão do sistema.

Como exemplo, o domínio de gestão de emergências define os seus tipos de mensagem através de um enum dedicado:

```
class EmergencyMsg(str, Enum):
    EMERGENCY_ALERT = "emergency_alert"
    EMERGENCY_ACK = "emergency_acknowledged"
    EMERGENCY_STATUS_REQ = "emergency_status"
    EMERGENCY_STATUS = "emergency_status"
    EMERGENCY_RESOLVED = "emergency_resolved"
```

3.2. Comunicação

A comunicação entre agentes ocorre em fluxos lógicos orientados por eventos, variando de acordo com o papel que cada agente assume em determinado momento. Estes fluxos refletem padrões clássicos de interação em sistemas multiagente:

- **Modelo Produtor–Consumidor:** utilizado no domínio de monitorização, onde agentes sensores enviam periodicamente dados biométricos para agentes de análise, que processam a informação e avaliam riscos de forma contínua;
- **Modelo de Delegação:** observado no domínio de gestão de emergências, no qual o agente central recebe alertas e delega tarefas específicas, como triagem ou despacho de meios de transporte, a outros agentes especializados;
- **Modelo de Negociação e Confirmação:** aplicado na interação com unidades hospitalares e agentes de transporte, onde pedidos de admissão ou despacho podem ser aceites ou recusados com base na disponibilidade atual.

Estes padrões podem ser observados diretamente nos fluxos de comunicação implementados. Por exemplo, no domínio de gestão de emergências, o agente central delega a triagem clínica a um agente especializado:

```
triage_msg = build_spade_message(
    to=TRIAGE_JID,
    performative=Performative.REQUEST,
    msg_type=EmergencyMsg.TRIAGE_CASE,
    payload=triage_payload,
)
```

E, em resposta:

```
reply = build_spade_message(
    to=str(msg.sender),
    performative=Performative.INFORM,
    msg_type=EmergencyMsg.TRIAGE_RESULT,
    payload=result_payload,
)
```

Esta diversidade de padrões permite que múltiplos fluxos de execução ocorram em paralelo, sem necessidade de coordenação centralizada rígida, promovendo a autonomia e cooperação entre agentes.

3.3. *Templates*

Para garantir eficiência e evitar o processamento de mensagens irrelevantes, os agentes recorrem a *templates* como mecanismo de filtragem na recepção de mensagens. Estes *templates* permitem que cada comportamento reaja apenas a mensagens com determinadas características, como a *performative* ou o tipo de mensagem associado.

No sistema desenvolvido, os *templates* são definidos através da associação de comportamentos a tipos específicos de mensagem, como exemplificado no agente central de emergências:

```
tpl = Template()
tpl.set_metadata("type", EmergencyMsg.EMERGENCY_ALERT.value)
self.add_behaviour(ReceiveEmergencyAlertsBehaviour(), tpl)
```

Esta abordagem possibilita que um mesmo agente execute vários comportamentos em simultâneo, cada um responsável por um fluxo de comunicação distinto, sem interferência entre eles. Como resultado, o sistema suporta concorrência, escalabilidade e múltiplos diálogos ativos em paralelo, mantendo a clareza e organização do processamento interno de cada agente.

4. Domínios do Sistema

4.1. Gestão de Emergências

O subdomínio de Gestão de Emergências constitui o núcleo operacional do sistema, que se encarrega da detecção, avaliação, coordenação e resolução de situações médicas. Este subdomínio inclui um conjunto de agentes especializados que cooperam de forma autónoma e distribuída para assegurar uma resposta rápida, consistente e adaptativa a eventos críticos.

Nos subcapítulos que se seguem, apresentamos uma análise detalhada de cada agente e os seus comportamentos.

4.1.1. EmergencyCenterAgent

Este agente desempenha o papel central de coordenação neste subdomínio. É responsável por **manter o estado global** das emergências ativas e por **orquestrar a interação** entre pacientes, triagem, meios de transporte e hospitais.

Behaviours

1. ReceiveEmergencyAlertsBehaviour

Este *behaviour* recebe mensagens do tipo `EMERGENCY_ALERT` com *performative* `REQUEST`. Ao receber um alerta, o agente:

- Regista a emergência internamente;
- Envia uma confirmação ao paciente (`EMERGENCY_ACK` , *performative* `INFORM`);
- Solicita a realização da triagem ao agente de triagem (`TRIAGE_CASE` , *performative* `REQUEST`).

2. HandleTriageResultBehaviour

Responsável por processar os resultados da triagem (`TRIAGE_RESULT` , *performative* `INFORM`). Com base nessa informação, o agente:

- Seleciona o hospital de destino utilizando uma política *round-robin*;
- Escolhe o meio de transporte adequado (ambulância ou helicóptero);
- Despacha o transporte (`TRANSPORT_DISPATCH` , *performative* `REQUEST`);
- Informa o paciente que o transporte foi acionado (`TRANSPORT_DISPATCHED` , *performative* `INFORM`).

3. HandleUpdatesBehaviour

Este *behaviour* gere todos os eventos subsequentes relacionados com a emergência, incluindo:

- Aceitação ou recusa de transporte (`TRANSPORT_ACCEPT` , `TRANSPORT_REFUSE`);
- Admissão ou recusa hospitalar (`PATIENT_ADMITTED` , `PATIENT_REFUSED`);
- Reencaminhamento para outro hospital em caso de lotação;
- Encerramento da emergência quando esta é resolvida, notificando o paciente (`EMERGENCY_RESOLVED` , *performative* `INFORM`).

4. HandleStatusQueriesBehaviour

Permite responder a pedidos de estado de uma emergência (`EMERGENCY_STATUS_REQ` , *performative* `REQUEST`), devolvendo informação atualizada sobre a ocorrência (`EMERGENCY_STATUS` , *performative* `INFORM`).

5. CenterReportBehaviour

Behaviour periódico responsável por reportar métricas globais, como o número de emergências ativas.

4.1.2. TriageAgent

Este agente é responsável pela avaliação clínica inicial das emergências, funcionando como um agente especialista na priorização e classificação dos casos.

Behaviours

1. ReceiveTriageCaseBehaviour

- Recebe pedidos de triagem (`TRIAGE_CASE` , *performative* `REQUEST`) enviados pelo centro de emergências. Este *behaviour*:
- Analisa gravidade, descrição clínica e sinais vitais;
- Determina o meio de transporte mais adequado;
- Identifica a especialidade médica necessária;
- Envia o resultado da triagem ao centro (`TRIAGE_RESULT` , *performative* `INFORM`).

2. TriageReportBehaviour

Behaviour periódico que mantém estatísticas sobre o número total de casos avaliados.

4.1.3. AmbulanceAgent

Este agente representa uma ambulância disponível no sistema, responsável pelo transporte terrestre de pacientes.

Behaviours

1. AmbulanceHandleDispatchBehaviour

Recebe pedidos de despacho (`TRANSPORT_DISPATCH` , *performative* `REQUEST`). Dependendo do seu estado:

- Aceita o transporte (`TRANSPORT_ACCEPT` , *performative* `CONFIRM`);
- Recusa se estiver ocupado (`TRANSPORT_REFUSE` , *performative* `REFUSE`);
- Notifica o hospital da chegada iminente do paciente (`PATIENT_INCOMING` , *performative* `INFORM`).

2. AmbulanceHospitalReplyBehaviour

Processa respostas do hospital (`PATIENT_ADMITTED` ou `PATIENT_REFUSED`). Este *behaviour*:

- Propaga a decisão ao centro de emergências;
- Conclui o transporte quando o paciente é admitido;
- Liberta a ambulância para novas ocorrências.

3. AmbulanceReportBehaviour

Behaviour periódico que reporta o estado da ambulância e o número de serviços realizados.

4.1.4. HelicopterAgent

Este agente tem um funcionamento semelhante ao da ambulância, representando um meio de transporte aéreo, geralmente utilizado em situações de maior gravidade ou urgência.

Behaviours

1. HelicopterHandleDispatchBehaviour

Aceita ou recusa pedidos de transporte aéreo e comunica com o hospital.

2. HelicopterHospitalReplyBehaviour

Trata admissões ou recusas hospitalares e encerra o serviço.

3. HelicopterReportBehaviour

Reporta métricas de utilização e disponibilidade.

As *performatives* utilizadas seguem o mesmo padrão do **AmbulanceAgent** (`REQUEST` , `CONFIRM` , `REFUSE` , `INFORM`).

4.1.5. HospitalAgent

Este agente representa uma unidade hospitalar com capacidade limitada para admissão de pacientes.

Behaviours

1. HospitalHandleIncomingBehaviour

Recebe notificações de pacientes a caminho (`PATIENT_INCOMING` , *performative* `INFORM`). O agente:

- Admite o paciente se houver capacidade (`PATIENT_ADMITTED` , *performative* `CONFIRM`);
- Recusa caso contrário (`PATIENT_REFUSED` , *performative* `REFUSE`).

2. HospitalSpecialistFlowBehaviour

Verifica periodicamente se algum paciente admitido necessita de uma especialidade específica e, se necessário, solicita um especialista externo (`SPECIALIST_REQUEST` , *performative* `REQUEST`).

3. HospitalHandleTreatmentUpdateBehaviour

Recebe atualizações de tratamento (`TREATMENT_UPDATE` , *performative* `INFORM`) e associa-as ao respetivo paciente.

4. HospitalReportBehaviour

Behaviour periódico que reporta o número de pacientes admitidos face à capacidade máxima.

4.2. Suporte ao Paciente

O subdomínio de Suporte ao Paciente é responsável pela interação direta entre o utilizador e o sistema, funcionando como o principal ponto de entrada para pedidos médicos. Este domínio abstrai a complexidade interna do sistema, oferecendo ao paciente uma interface unificada para a submissão de solicitações, receção de respostas e acompanhamento do estado dos seus pedidos.

Nos subcapítulos que se seguem, apresentamos uma análise detalhada do agente e os seus comportamentos.

4.2.1. ChatbotPatientAgent

Este agente representa um agente de interface conversacional que simula a interação direta com o paciente. É responsável por interpretar o pedido inicial do utilizador, realizar uma triagem preliminar e encaminhar automaticamente a solicitação para o domínio apropriado, mantendo posteriormente o utilizador informado sobre a evolução do pedido.

Este agente não toma decisões clínicas finais, mas atua como um mediador inteligente entre o utilizador e os restantes agentes do sistema.

Behaviours

1. ChatbotTriageRouteBehaviour

Este *behaviour* é executado uma única vez no arranque do agente e simula a recolha inicial de informação fornecida pelo utilizador. Com base numa heurística simples, o pedido é classificado como urgente ou não urgente.

Situações urgentes

Quando são detetados sintomas críticos, o agente gera um alerta de emergência e envia-o para o centro de emergências:

- **Mensagem:** EMERGENCY_ALERT
- **Performative:** REQUEST

Neste caso, o **ChatbotPatientAgent** passa a acompanhar o estado da emergência até à sua resolução.

Situações não urgentes

Para pedidos de menor gravidade, o agente cria um pedido de marcação de consulta:

- **Mensagem:** APPOINTMENT_REQUEST
- **Performative:** REQUEST

O pedido é encaminhado para o domínio de agendamentos, sendo selecionada uma especialidade médica de forma heurística.

Este *behaviour* permite desacoplar a interação com o utilizador da lógica interna dos domínios clínicos, promovendo modularidade e escalabilidade.

2. ChatbotListenRepliesBehaviour

É responsável por escutar e processar todas as mensagens de resposta enviadas pelos restantes domínios do sistema. Este *behaviour* funciona de forma cíclica e permite ao agente atualizar o seu estado interno e informar o utilizador sobre a evolução do pedido.

No contexto de consultas médicas, o agente reage às seguintes mensagens:

- APPOINTMENT_QUEUED (INFORM) – pedido registado e em fila;
- APPOINTMENT_SCHEDULED (INFORM) – consulta agendada;
- APPOINTMENT_COMPLETED (INFORM) – consulta concluída.

No contexto de emergências, o agente processa:

- EMERGENCY_ACK (INFORM) – confirmação da receção do alerta;
- TRANSPORT_DISPATCHED (INFORM) – meio de socorro acionado;
- EMERGENCY_RESOLVED (INFORM) – emergência resolvida.

Após a conclusão do processo (consulta concluída ou emergência resolvida), o agente termina a sua execução, refletindo o encerramento da interação com o paciente.

4.3. Agendamentos

O subdomínio de Agendamentos é responsável pela gestão de pedidos de consultas médicas e pela atribuição dinâmica de médicos especialistas, atuando como um intermediário inteligente entre os pacientes (ou unidades hospitalares) e os profissionais de saúde disponíveis. Este domínio assegura que cada pedido é encaminhado para um médico com a especialidade adequada, tendo em conta a disponibilidade dos recursos e a ordem de chegada das solicitações.

Para além dos pedidos de consulta iniciados pelo paciente, este subdomínio desempenha também um papel fundamental no suporte às emergências, permitindo que unidades hospitalares solicitem médicos especialistas sempre que um paciente admitido necessite de acompanhamento específico. Desta forma, o domínio de agendamentos contribui para uma resposta médica integrada e contínua ao longo de todo o sistema.

4.3.1. SchedulerAgent

Este agente é o agente central do subdomínio de agendamentos, responsável por receber pedidos, gerir filas por especialidade, monitorizar a disponibilidade dos médicos e efetuar a atribuição dinâmica de consultas.

Internamente, o agente mantém filas de pedidos pendentes organizadas por especialidade, listas de médicos disponíveis por especialidade e um registo das consultas ativas e concluídas.

Behaviours

1. ReceiveAppointmentRequestsBehaviour

Este *behaviour* recebe pedidos de consulta enviados por agentes de suporte ao paciente:

- **Mensagem:** `APPOINTMENT_REQUEST`
- **Performative:** `REQUEST`

Após a receção do pedido, o agente regista o pedido na fila correspondente à especialidade, responde ao paciente com uma confirmação de que o pedido foi colocado em fila (`APPOINTMENT_QUEUED` , *performative* `INFORM`) e tenta imediatamente atribuir um médico disponível.

2. ReceiveSpecialistRequestsBehaviour

Este *behaviour* trata pedidos de médicos especialistas enviados por unidades hospitalares no contexto de emergências:

- **Mensagem:** `SPECIALIST_REQUEST`
- **Performative:** `REQUEST`

O pedido é tratado de forma análoga a uma consulta, sendo colocado na fila da especialidade correspondente, mas identificado como um pedido de modo especialista, associado a uma emergência específica.

3. HandleDoctorAvailabilityBehaviour

Responsável por receber notificações de disponibilidade dos médicos:

- **Mensagem:** `DOCTOR_AVAILABLE`
- **Performative:** `INFORM`

Sempre que um médico se torna disponível, este *behaviour* atualiza as listas internas e tenta atribuir automaticamente consultas pendentes da respetiva especialidade.

4. HandleDoctorDoneBehaviour

Este *behaviour* processa a conclusão de uma consulta ou intervenção especializada:

- **Mensagem:** `DOCTOR_DONE`
- **Performative:** `INFORM`

Após a conclusão, a consulta é marcada como concluída, o registo ativo é removido e são atualizadas métricas globais do sistema.

5. SchedulerReportBehaviour

Behaviour periódico responsável pela recolha e apresentação de métricas globais do subdomínio, incluindo número de pedidos, consultas agendadas, concluídas e recursos disponíveis.

4.3.2. DoctorAgent

Este agente representa um médico especialista no sistema. Cada agente está associado a uma especialidade médica específica e gere de forma autónoma a sua disponibilidade e o ciclo de vida das consultas que lhe são atribuídas.

Behaviours

1. DoctorRegisterBehaviour

Executado no arranque do agente, este *behaviour* informa o **SchedulerAgent** da disponibilidade inicial do médico:

- **Mensagem:** DOCTOR_AVAILABLE
- **Performative:** INFORM

Esta abordagem permite uma descoberta dinâmica de recursos e facilita a escalabilidade do sistema.

2. DoctorHandleAssignmentBehaviour

Este *behaviour* trata a atribuição de consultas ou pedidos de especialidade:

- **Mensagem:** DOCTOR_ASSIGN
- **Performative:** REQUEST

Caso o médico esteja disponível, o agente aceita a atribuição, informa o paciente (ou hospital) de que a consulta foi agendada (APPOINTMENT_SCHEDULED , *performative* INFORM), simula a realização da consulta ou intervenção especializada e notifica o **SchedulerAgent** da conclusão (DOCTOR_DONE , *performative* INFORM) e volta a anunciar a sua disponibilidade.

No caso de pedidos de modo especialista, o médico envia ainda uma atualização clínica para o hospital:

- **Mensagem:** TREATMENT_UPDATE
- **Performative:** INFORM

3. DoctorReportBehaviour

Behaviour periódico que apresenta métricas locais do agente, como o número total de consultas realizadas e o estado atual de disponibilidade.

4.4. Monitorização

O subdomínio de Monitorização é responsável pela observação contínua do estado fisiológico dos pacientes, através da recolha e análise de sinais vitais em tempo quase real. Este domínio atua como um mecanismo preventivo e reativo, permitindo a deteção precoce de situações clínicas anómalas e a ativação automática de processos de resposta quando são identificadas condições críticas.

Para evitar alarmes excessivos e sobrecarga dos serviços de emergência, este subdomínio incorpora mecanismos de **filtragem de ruído e controlo de frequência de alertas** (*cooldown*), garantindo que apenas situações relevantes originam alertas clínicos. Sempre que uma condição crítica é confirmada, o domínio de monitorização comunica diretamente com o domínio de **Gestão de Emergências**, desencadeando processos de triagem e resposta médica. Paralelamente, mantém um histórico recente de sinais vitais que pode ser utilizado para suporte à decisão clínica e análise de tendências.

4.4.1. MonitoringAgent

Este agente é o agente central do subdomínio de monitorização, responsável por receber atualizações de sinais vitais, manter um histórico temporal por paciente e decidir quando deve ser criado um alerta de emergência.

Este agente implementa uma lógica de análise baseada em regras heurísticas, complementada por um mecanismo de *debounce* temporal, assegurando que alertas repetidos para o mesmo paciente não são gerados de forma excessiva.

Behaviours

1. MonitoringReceiveVitalsBehaviour

Este *behaviour* recebe atualizações periódicas de sinais vitais provenientes dos sensores:

- **Mensagem:** VITALS_UPDATE
- **Performative:** INFORM

Ao receber uma atualização, o agente armazena os sinais vitais num histórico limitado por paciente, avalia se os valores indicam uma condição crítica, calcula um score de severidade com base em parâmetros como frequência cardíaca, saturação de oxigénio e pressão arterial e verifica se deve ser gerado um novo alerta, tendo em conta regras de cooldown e agravamento do estado clínico.

Quando é decidido que a situação é crítica e relevante, o agente cria automaticamente um alerta de emergência:

- **Mensagem:** EMERGENCY_ALERT
- **Performative:** REQUEST

Este alerta é enviado para o centro de emergências, iniciando o fluxo completo de gestão de emergência.

2. MonitoringReportBehaviour

Behaviour periódico responsável por reportar métricas globais do domínio de monitorização, incluindo:

- número total de **atualizações de sinais vitais** recebidas;
- número de **alertas** gerados;
- número de **pacientes monitorizados** ativamente.

Este *behaviour* permite observar o comportamento do sistema ao longo do tempo e facilita a análise do desempenho do domínio.

4.4.2. SensorAgent

Este agente representa um sensor físico ou lógico associado a um paciente, responsável pela recolha periódica de sinais vitais e pelo seu envio ao sistema de monitorização. Cada instância deste agente está associada a um único paciente, permitindo uma monitorização distribuída e escalável.

Behaviours

1. SensorSendVitalsBehaviour

Este *behaviour* é executado periodicamente e simula a geração de sinais vitais com uma distribuição realista:

- a maioria das amostras representa valores normais ou ligeiramente alterados;
- uma pequena percentagem corresponde a episódios críticos, permitindo testar o comportamento do sistema.

Os sinais vitais são enviados ao **MonitoringAgent** através de:

- **Mensagem:** VITALS_UPDATE
- **Performative:** INFORM

Esta abordagem permite desacoplar a geração de dados da sua análise, reforçando o carácter distribuído do sistema.

5. Políticas de Decisão

Este capítulo sumariza as principais regras e heurísticas implementadas no sistema, que orientam decisões de triagem, ativação de emergências, encaminhamento e gestão de recursos (transporte e capacidade hospitalar). Estas políticas permitem justificar o comportamento observado nos fluxos e nos testes executados.

5.1. Triagem no Suporte ao Paciente (*ChatbotPatientAgent*)

O *ChatbotPatientAgent* atua como ponto de entrada para pedidos do utilizador e aplica uma triagem preliminar para distinguir situações potencialmente críticas de pedidos não urgentes. Esta decisão baseia-se numa lista curta de palavras-chave associadas a sintomas de maior gravidade (por exemplo, dor no peito ou dificuldade respiratória). Quando o pedido é classificado como urgente, o *chatbot* encaminha-o para o domínio de gestão de emergências; caso contrário, o pedido segue para o domínio de agendamentos, iniciando um processo normal de marcação de consulta.

Excerto: triagem do *chatbot* com *urgent_keywords* e *is_urgent*

```
urgent_keywords = {"dor no peito", "dificuldade respiratória", "desmaio",  
"hemorragia"}  
is_urgent = symptoms in urgent_keywords
```

5.2. Critérios Clínicos de Triagem (*TriageAgent*)

Após a abertura formal de uma ocorrência de emergência, a decisão clínica é assumida pelo *TriageAgent*. A triagem tem dois objetivos práticos: (i) atribuir um nível de prioridade e severidade operacional, e (ii) devolver recomendações que permitam ao centro de emergência agir rapidamente, nomeadamente o tipo de transporte mais adequado e a especialidade clínica a acionar.

No que diz respeito ao transporte, o sistema privilegia a rapidez em casos de gravidade elevada, optando por helicóptero quando a severidade ultrapassa um limiar alto ou quando existem sinais claros de compromisso respiratório. Caso contrário, é selecionada uma ambulância como meio padrão. Em paralelo, a triagem procura também inferir a especialidade necessária de forma simples e interpretável, combinando indicadores fisiológicos (por exemplo, valores extremos de frequência cardíaca ou saturação) com pistas presentes na descrição do incidente.

Excerto: seleção de transporte com limiares de *severity/spo2*

```
transport = TransportType.HELICOPTER if (severity >= 9 or spo2 < 88) else  
TransportType.AMBULANCE
```

Excerto: determinação da especialidade requerida a partir de *description/vitals*

```
if "peito" in description or hr > 130:  
    specialty = MedicalSpecialty.CARDIOLOGY  
elif "respirat" in description or spo2 < 90:  
    specialty = MedicalSpecialty.GENERAL  
else:  
    specialty = MedicalSpecialty.GENERAL
```

5.3. Detecção Automática de Situações Críticas (*MonitoringAgent*)

O *MonitoringAgent* é responsável por analisar as atualizações periódicas dos sinais vitais recebidos do *SensorAgent* e decidir quando existe risco suficiente para desencadear um alerta de emergência. Para isso, o sistema recorre a uma lógica de avaliação que combina dois níveis: uma verificação direta de condições críticas (ex.: saturação muito baixa ou taquicardia severa) e um mecanismo de pontuação de risco que ajuda a quantificar o desvio face ao normal.

Para evitar alarmes repetidos e desnecessários, especialmente em ambientes com amostragem frequente, a geração de alertas inclui ainda um mecanismo de contenção, através de *cooldown* e *debounce*. Na prática, isto impede que o mesmo paciente origine sucessivos alertas com poucos segundos de diferença, exceto quando há uma deterioração real e significativa do estado clínico.

Excerto: função/condição “critical” baseada em *thresholds*

```
def is_critical(vitals) -> bool:
    ...
    return (spo2 < 90) or (hr > 130) or (syst < 90)
```

Excerto: cálculo do *score* de risco

```
score = 0
if hr > 130:
    score += 2
if hr > 150:
    score += 1

if spo2 < 90:
    score += 3
if spo2 < 85:
    score += 1

if syst < 90:
    score += 3
if syst < 80:
    score += 1

score = min(score, 10)
```

Excerto: regra de *cooldown/debounce* (tempo + agravamento do *score*)

```
if delta >= self.alert_cooldown_seconds or score >= active.last_score + 2:
    self.active_alerts[patient_id] = ActiveAlert(
        emergency_id=emergency_id,
        last_alert_ts=now_ts,
        last_score=score,
    )
    return True
return False
```

5.4. Gestão de Recursos e Resiliência a Indisponibilidades

Como o sistema modela recursos finitos (meios de transporte e capacidade hospitalar), foram introduzidas regras explícitas para lidar com indisponibilidades sem comprometer o fluxo global. Do lado dos transportes, um pedido de despacho pode ser recusado quando o meio se encontra ocupado, permitindo ao centro de emergência reagir e selecionar uma alternativa. Do lado hospitalar, a admissão depende da capacidade corrente; quando o hospital atinge o limite definido, responde com recusa, sinalizando explicitamente a impossibilidade de receber mais pacientes naquele momento.

Um aspeto relevante é o comportamento do *EmergencyCenterAgent* perante recusas: em vez de falhar silenciosamente, o sistema tenta reencaminhar o paciente para outra unidade hospitalar, mantendo o mesmo processo de emergência ativo. Se não existir alternativa viável, a ocorrência é encerrada com um estado informativo, garantindo que o sistema permanece consistente e que não ficam emergências “presas” em estado indefinido.

Excerto: recusa do transporte quando ocupado (*REFUSE + reason*)

Ambulância

```
if not self.agent.available:
    refuse_payload = {"reason": "ambulance_busy", "timestamp": get_current_time()}
    refuse = build_spade_message(
        to=sender_center,
        performative=Performative.REFUSE,
        msg_type=EmergencyMsg.TRANSPORT_REFUSE,
        payload=refuse_payload,
    )
    await self.send(refuse)
    return
```

Helicóptero

```
if not self.agent.available:
    refuse_payload = {"reason": "helicopter_busy", "timestamp": get_current_time()}
    refuse = build_spade_message(
        to=sender_center,
        performative=Performative.REFUSE,
        msg_type=EmergencyMsg.TRANSPORT_REFUSE,
        payload=refuse_payload,
    )
    await self.send(refuse)
    return
```

Excerto: verificação de capacidade hospitalar e recusa por lotação

```
def has_capacity(self) -> bool:
    return len(self.agent.admitted) < self.agent.capacity
```

```
if self.agent.has_capacity():
    # CONFIRM patient_admitted
else:
    # REFUSE patient_refused (hospital_full)
```

Excerto: estratégia de reroute / fallback quando hospital recusa

```
if payload.get("status") == "hospital_full":
    new_hospital = self.agent.pick_hospital(exclude={hospital_jid})
    if new_hospital:
        reroute = build_spade_message(
            to=transport_jid,
            performative=Performative.REQUEST,
            msg_type=EmergencyMsg.TRANSPORT_REROUTE,
            payload=reroute_payload,
        )
        await self.send(reroute)
    else:
        info_payload = {"status": "no_hospital_available", "timestamp":
            get_current_time()}
```

5.5. Continuidade de Cuidados e Encaminhamento para Especialista

Após a admissão, o *HospitalAgent* pode desencadear automaticamente a continuidade de cuidados, solicitando uma consulta especializada quando a triagem indica que a especialidade necessária não é geral. Esta decisão permite ligar o domínio de emergências ao domínio de agendamentos de forma coerente: o pedido hospitalar é tratado como um novo processo de escalonamento, atribuído a um *DoctorAgent* compatível e concluído com uma atualização clínica de tratamento. Com isto, o sistema suporta não apenas a resposta imediata à emergência, mas também o encaminhamento pós-admissão que representa uma fase comum em contextos reais de cuidados de saúde.

Excerto: *specialist_request* (*HospitalAgent* → *SchedulerAgent*) e tratamento/conclusão

```
if specialty == MedicalSpecialty.GENERAL.value:
    continue
if rec.get("specialist_requested"):
    continue

msg = build_spade_message(
    to=SCHEDULER_JID,
    performative=Performative.REQUEST,
    msg_type=SpecialistMsg.SPECIALIST_REQUEST,
    payload=payload,
)
await self.send(msg)
rec["specialist_requested"] = True
```

```

if mode == "specialist" and emergency_id:
    update_payload = {
        "emergency_id": emergency_id,
        "doctor_jid": str(self.agent.jid),
        "specialty": specialty,
        "note": "Specialist treatment completed",
        "timestamp": get_current_time(),
    }
    upd = build_spade_message(
        to=patient_jid, # aqui é o hospital_jid
        performative=Performative.INFORM,
        msg_type=SpecialistMsg.TREATMENT_UPDATE,
        payload=update_payload,
    )
    await self.send(upd)

```

6. Fluxos de Funcionamento

Esta secção descreve os principais fluxos de funcionamento considerados no desenvolvimento e validação do sistema. Cada fluxo corresponde a um cenário típico do domínio dos cuidados de saúde e envolve a interação coordenada entre múltiplos agentes pertencentes a diferentes domínios funcionais.

Os fluxos apresentados foram implementados e testados através de cenários de execução específicos, permitindo avaliar o comportamento distribuído do sistema, a troca de mensagens entre agentes e a correta aplicação das decisões autónomas em contexto realista.

6.1. Fluxo 1: Pedido de Consulta Médica

Este fluxo descreve o processo de marcação de uma consulta médica não urgente, iniciado por um paciente através do subsistema de suporte ao paciente.

1. O paciente interage com o *ChatbotPatientAgent*, indicando a intenção de marcar uma consulta médica;
2. O *ChatbotPatientAgent* realiza uma triagem inicial do pedido e identifica que se trata de uma situação não urgente;
3. O pedido é encaminhado para o *SchedulerAgent* através de uma mensagem `APPOINTMENT_REQUEST` com *performative* `REQUEST` ;
4. O *SchedulerAgent* regista o pedido numa fila associada à especialidade solicitada e informa o paciente de que o pedido foi colocado em espera, enviando uma mensagem `APPOINTMENT_QUEUED` (`INFORM`);
5. Quando um *DoctorAgent* com a especialidade adequada se encontra disponível, o *SchedulerAgent* envia um pedido de atribuição através de uma mensagem `DOCTOR_ASSIGN` (`REQUEST`);
6. O *DoctorAgent* aceita a atribuição, informa o paciente de que a consulta foi agendada (`APPOINTMENT_SCHEDULED` , `INFORM`) e simula a execução da consulta;
7. Após a conclusão da consulta, o *DoctorAgent* notifica o *SchedulerAgent* (`DOCTOR_DONE` , `INFORM`) e volta a anunciar a sua disponibilidade.

Este fluxo valida o funcionamento do mecanismo de agendamento autónomo, bem como a coordenação entre agentes de suporte ao paciente, escalonamento e médicos especialistas.

Exemplo de mensagens trocadas:

```
ChatbotPatientAgent → SchedulerAgent : APPOINTMENT_REQUEST (REQUEST)
SchedulerAgent → ChatbotPatientAgent : APPOINTMENT_QUEUED (INFORM)
SchedulerAgent → DoctorAgent          : DOCTOR_ASSIGN (REQUEST)
DoctorAgent → ChatbotPatientAgent      : APPOINTMENT_SCHEDULED (INFORM)
DoctorAgent → SchedulerAgent           : DOCTOR_DONE (INFORM)
```

6.2. Fluxo 2: Emergência Detetada por Monitorização

Este fluxo representa um cenário em que uma situação crítica é detetada automaticamente através da monitorização contínua de sinais vitais.

1. O *SensorAgent* gera periodicamente sinais vitais e envia-os ao *MonitoringAgent* através de mensagens `VITALS_UPDATE` (`INFORM`);
2. O *MonitoringAgent* analisa os dados recebidos, mantendo um histórico recente e avaliando a severidade dos sinais vitais;
3. Ao identificar valores fora dos intervalos de segurança e após validação por mecanismos de *cooldown* e *debounce*, o *MonitoringAgent* gera um alerta de emergência;

4. O alerta é enviado ao *EmergencyCenterAgent* através de uma mensagem `EMERGENCY_ALERT` (`REQUEST`);
5. O *EmergencyCenterAgent* registra a emergência e solicita uma avaliação clínica ao *TriageAgent* (`TRIAGE_CASE`, `REQUEST`);
6. O *TriageAgent* analisa a situação e devolve o resultado da triagem (`TRIAGE_RESULT`, `INFORM`);
7. Com base na triagem, o *EmergencyCenterAgent* seleciona o meio de transporte e a unidade hospitalar e envia um pedido de despacho (`TRANSPORT_DISPATCH`, `REQUEST`);
8. O agente de transporte selecionado aceita a missão e notifica a unidade hospitalar da chegada iminente do paciente (`PATIENT_INCOMING`, `INFORM`);
9. O *HospitalAgent* decide aceitar ou recusar a admissão do paciente com base na sua capacidade.

Exemplo de mensagens trocadas:

```

SensorAgent      → MonitoringAgent      : VITALS_UPDATE (INFORM)
MonitoringAgent   → EmergencyCenterAgent : EMERGENCY_ALERT (REQUEST)
EmergencyCenterAgent → TriageAgent       : TRIAGE_CASE (REQUEST)
TriageAgent       → EmergencyCenterAgent : TRIAGE_RESULT (INFORM)
EmergencyCenterAgent → TransportAgent    : TRANSPORT_DISPATCH (REQUEST)
TransportAgent     → EmergencyCenterAgent : TRANSPORT_ACCEPT (CONFIRM)
TransportAgent     → HospitalAgent       : PATIENT_INCOMING (INFORM)
HospitalAgent      → TransportAgent      : PATIENT_ADMITTED (CONFIRM)
EmergencyCenterAgent → PatientAgent      : EMERGENCY_RESOLVED (INFORM)

```

Este fluxo demonstra a integração entre os domínios de monitorização e gestão de emergências, bem como a capacidade de resposta automática e preventiva do sistema.

6.3. Fluxo 3: Emergência Reportada por Paciente

Neste cenário, a emergência é reportada diretamente pelo paciente, sem intervenção do subsistema de monitorização.

1. O *PatientAgent* envia um alerta de emergência ao *EmergencyCenterAgent* (`EMERGENCY_ALERT`, `REQUEST`);
2. O *EmergencyCenterAgent* registra a ocorrência e confirma a receção ao paciente (`EMERGENCY_ACK`, `INFORM`);
3. O centro solicita uma avaliação clínica ao *TriageAgent* (`TRIAGE_CASE`, `REQUEST`);
4. Após receber o resultado da triagem (`TRIAGE_RESULT`, `INFORM`), o *EmergencyCenterAgent* despacha um meio de transporte adequado;
5. O agente de transporte aceita o pedido (`TRANSPORT_ACCEPT`, `CONFIRM`) e inicia o processo de transporte;
6. O *HospitalAgent* é notificado da chegada do paciente e responde com a sua decisão de admissão (`PATIENT_ADMITTED` ou `PATIENT_REFUSED`);
7. Após a resolução da emergência, o *EmergencyCenterAgent* encerra o processo e informa o paciente (`EMERGENCY_RESOLVED`, `INFORM`).

Exemplo de mensagens trocadas:

```

PatientAgent      → EmergencyCenterAgent : EMERGENCY_ALERT (REQUEST)
EmergencyCenterAgent → PatientAgent      : EMERGENCY_ACK (INFORM)
EmergencyCenterAgent → TriageAgent       : TRIAGE_CASE (REQUEST)
TriageAgent       → EmergencyCenterAgent : TRIAGE_RESULT (INFORM)
EmergencyCenterAgent → TransportAgent    : TRANSPORT_DISPATCH (REQUEST)
TransportAgent     → EmergencyCenterAgent : TRANSPORT_ACCEPT (CONFIRM)
TransportAgent     → HospitalAgent       : PATIENT_INCOMING (INFORM)
HospitalAgent      → TransportAgent      : PATIENT_ADMITTED (CONFIRM)
EmergencyCenterAgent → PatientAgent      : EMERGENCY_RESOLVED (INFORM)

```

Este fluxo valida o comportamento reativo do sistema perante emergências reportadas manualmente e a correta coordenação entre agentes distribuídos.

6.4. Fluxo 4: Encaminhamento Hospitalar e Continuidade de Cuidados

Este fluxo ilustra a continuidade de cuidados após a admissão hospitalar de um paciente, garantindo acompanhamento especializado quando necessário.

1. Após a admissão do paciente, o *HospitalAgent* avalia a necessidade de acompanhamento por um médico especialista;
2. Caso seja identificada essa necessidade, o *HospitalAgent* envia um pedido de especialista ao *SchedulerAgent* (`SPECIALIST_REQUEST` , `REQUEST`);
3. O pedido é tratado como um processo de agendamento, sendo atribuído um *DoctorAgent* com a especialidade adequada;
4. O *DoctorAgent* realiza a intervenção especializada e envia uma atualização clínica ao hospital (`TREATMENT_UPDATE` , `INFORM`);
5. O *HospitalAgent* registra a atualização e integra a informação no processo clínico do paciente.

Exemplo de mensagens trocadas:

```
HospitalAgent    → SchedulerAgent : SPECIALIST_REQUEST (REQUEST)
SchedulerAgent   → DoctorAgent    : DOCTOR_ASSIGN (REQUEST)
DoctorAgent      → HospitalAgent  : TREATMENT_UPDATE (INFORM)
DoctorAgent      → SchedulerAgent  : DOCTOR_DONE (INFORM)
DoctorAgent      → SchedulerAgent  : DOCTOR_AVAILABLE (INFORM)
```

Este fluxo demonstra a articulação entre os domínios de emergências e agendamentos, assegurando continuidade e coerência no tratamento do paciente ao longo do sistema.

7. Análise Crítica

7.1. Resultados Obtidos

O sistema desenvolvido permitiu validar os principais fluxos funcionais definidos no âmbito do projeto, demonstrando coordenação correta entre agentes autónomos pertencentes a diferentes domínios do sistema.

As provas apresentadas neste capítulo foram retiradas do output do cenário `run_full_system_debug.py`.

Validação do subsistema de agendamentos

Os cenários de teste confirmaram o funcionamento do subsistema de agendamentos, no qual pedidos iniciados por pacientes via *ChatbotPatientAgent* são encaminhados para o *SchedulerAgent*, colocados em fila e atribuídos automaticamente a médicos disponíveis.

É possível observar (i) o pedido gerado pelo chatbot, (ii) a confirmação de enfileiramento, (iii) o agendamento por atribuição a um médico, e (iv) a conclusão do processo no escalonador:

```
[CHATBOT] TRIAGE NON-URGENT -> appointment_request: {... 'specialty': 'general' ...}
[CHATBOT] Received perf=inform type=appointment_queued payload={...}
[CHATBOT] Received perf=inform type=appointment_scheduled payload={... 'doctor_jid':
'doctor2@...' ...}
[SCHED] Completed appointment_id=APT-P-CHAT-7082 ... status='completed'
```

Adicionalmente, os *reports* periódicos do *Scheduler* evidenciam consistência do estado global do sistema (pedidos registados, consultas agendadas e concluídas), por exemplo:

```
[SCHED] Report: requests=3 scheduled=2 completed=2 pending=1 available_doctors=4
```

Este conjunto de evidências valida não só o fluxo funcional, como também a manutenção consistente de estado sob execução concorrente (pedidos e disponibilidade de médicos a variar durante o cenário).

Validação do domínio de gestão de emergências

No domínio de gestão de emergências, foi verificada resposta adequada tanto a emergências reportadas diretamente por paciente como a emergências detetadas automaticamente pela monitorização.

- **Emergência reportada por paciente**

Observa-se um ciclo completo: alerta do paciente, confirmação de receção, triagem, despacho de transporte, aceitação hospitalar e encerramento da emergência:

```
[PATIENT] REQUEST emergency_alert: {... 'severity': 6, 'source': 'patient'}
[CENTER] Received emergency_alert from=patient_emergency@...
[PATIENT] Received perf=inform type=emergency_acknowledged ...
[TRIAGE] INFORM triage_result: {... 'transport_type': 'ambulance',
'required_specialty': 'orthopedics' ...}
[CENTER] Dispatched transport=ambulance ...
[HOSPITAL] Incoming patient from=ambulance@...
[AMBULANCE] Hospital reply perf=confirm type=patient_admitted ...
[CENTER] Emergency resolved/closed (admitted) ...
[PATIENT] Received perf=inform type=emergency_resolved ...
```

Isto confirma a execução *end-to-end* do fluxo definido para emergências manuais, incluindo uso coerente de *performatives* (REQUEST , INFORM , CONFIRM) ao longo da interação.

- **Emergência detetada por monitorização**

O output mostra também a geração de alertas automáticos quando surgem sinais vitais críticos, incluindo evidência do mecanismo de *debounce* (evitar alertas repetidos demasiado próximos):

```
[MONITOR] REQUEST emergency_alert: {... 'severity': 10, 'source': 'monitoring'}  
[MONITOR] CRITICAL but debounced patient=P-MON score=9 ...
```

Após o alerta, o sistema executa novamente o ciclo de triagem e despacho, com seleção de transporte coerente com maior severidade (helicóptero) e encerramento após admissão:

```
[TRIAGE] INFORM triage_result: {... 'transport_type': 'helicopter',  
'required_specialty': 'cardiology' ...}  
[CENTER] Dispatched transport=helicopter ...  
[HOSPITAL] Incoming patient from=helicopter@...  
[HELICOPTER] Hospital reply perf=confirm type=patient_admitted ...  
[CENTER] Emergency resolved/closed (admitted) ...
```

Os relatórios periódicos reforçam ainda que o sistema mantém controlo do número de emergências ativas e casos de triagem processados:

```
[CENTER] Report: active_emergencies=0  
[TRIAGE] Report: total_cases=5
```

Continuidade de cuidados (especialista após admissão)

O cenário demonstra também a continuidade de cuidados após admissão hospitalar, com pedidos de especialista gerados pelo *HospitalAgent* e tratados via *SchedulerAgent*, culminando num *treatment_update*:

```
[HOSPITAL] REQUEST specialist_request emergency_id=... specialty=cardiology  
[SCHED] queued specialist_request emergency_id=... specialty=cardiology  
[SCHED] Assigned doctor=doctor@... appointment_id=SPECIALIST-...  
[HOSPITAL] treatment_update emergency_id=... payload={... 'note': 'Specialist  
treatment completed' ...}  
[SCHED] Completed appointment_id=SPECIALIST-... status='completed'
```

Isto valida a integração entre domínios de emergência e agendamentos, demonstrando reutilização do mesmo mecanismo de escalonamento para consultas de seguimento no contexto hospitalar.

Robustez da comunicação e execução concorrente

A comunicação entre agentes mostrou-se eficaz e robusta, suportando múltiplos diálogos em paralelo. Isto é visível no próprio cenário, onde coexistem:

- atualizações contínuas de monitorização (*Sensor* → *Monitoring*),
- uma consulta em execução (*Chatbot/Scheduler/Doctor*),
- e múltiplas emergências (*Patient/Center/Triage/Transport/Hospital*),

mantendo coerência no estado global reportado e sem bloqueio do sistema. Um indicador relevante é a presença de *reports* periódicos consistentes (contadores de emergências ativas, triagens processadas, admissões e consultas concluídas), mesmo com múltiplos eventos intercalados.

8. Conclusões

Em conclusão, o presente trabalho permitiu conceber e implementar com sucesso um sistema multiagente aplicado ao domínio dos cuidados de saúde, demonstrando a adequação desta abordagem para ambientes complexos, distribuídos e dinâmicos. A arquitetura modular adotada, suportada pela *framework* SPADE e por comunicação assíncrona baseada em princípios da norma FIPA-ACL, revelou-se eficaz na coordenação entre agentes autónomos, assegurando escalabilidade, robustez e flexibilidade do sistema. Os resultados obtidos evidenciam o correto funcionamento dos diferentes domínios funcionais: gestão de emergências, agendamentos, monitorização e suporte ao paciente, bem como a sua integração coerente ao longo de múltiplos fluxos clínicos realistas. Destaca-se ainda a capacidade do sistema responder tanto a situações críticas detetadas automaticamente como a pedidos iniciados por utilizadores, garantindo continuidade de cuidados e gestão eficiente de recursos. De forma global, o projeto cumpriu os objetivos propostos e constitui uma base sólida para futuras extensões.

9. Avaliação Por Pares

Bruno - 0

Fernando - 0

Lara - 0

Pedro - 0

Sara - 0